

Name: _____

CptS 355 Exam 2 (Fall 2026)

Write your full name on every page. Keep explanations concise; partial credit is available. Assume required headers and enclosing functions are supplied for C++; Rust statement excerpts are inside `main`. Assume the usual `vtable` implementation in C++, and safe Rust with sound libraries.

1. (10 pts) In C++, write the recursive descent parsing function `int parseValue()` for this grammar:

$$V \rightarrow n \mid (V)$$

Here n represents a natural-number token. The function consumes one V and returns its numeric value, so parsing `((7))` returns 7. Use these supplied helpers:

```
std::string peek(); // next token's exact text; "" at end of input
bool isNat(const std::string& token); // true for one or more digits
int readNat(); // consume a natural-number token and return its value
void expect_token(const std::string& text); // consume text or report error
void syntaxError(); // report a syntax error; does not return
```

For example, `peek()` returns `"7"`, `"("`, or `)"`. Use `isNat` to recognize a number. Consume the matching parentheses around a recursively parsed V , and reject other starting tokens. All numbers fit in an `int`; the caller checks for leftover input.

2. (5 pts) Consider these Rust snippets independently. Which compiles, and which is rejected? State the borrowing rule involved. For the rejected snippet, describe how to perform the two increments one after the other without keeping both mutable borrows active together.

```
// A
let n = 1;
let a = &n;
let b = &n;
println!("{}", a, b);
```

```
// B
let mut n = 1;
let a = &mut n;
let b = &mut n;
*a += 1;
*b += 1;
println!("{}", n);
```

3. (4 pts) In C++, explain when `p` becomes dangling and why printing 7 is not guaranteed. Repair the function and caller by returning an integer by value, without heap allocation. Do not predict undefined behavior.

```
int* makeNumber() {
    int n = 7;
    return &n;
}
int* p = makeNumber();
std::cout << *p << "\n";
```

4. (7 pts) Prepare this grammar for recursive descent parsing; n is a natural-number token:

$$E \rightarrow E + T \mid E - T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow n \mid (E)$$

Identify both immediately left-recursive nonterminals. Suppose you wrote a recursive descent parsing function `parseE()` directly from the rule for E . Why would its first recursive call fail to make progress? Remove E 's immediate left recursion using one new nonterminal; do not rewrite T . Draw the AST for $2 + 3 * 4$ (no grammar-only nodes) and give its value.

5. (7 pts) In C++, each `Box` owns its allocated integer. After the default copy `Box b = a`, do the two pointers designate the same allocation or different allocations? What bug could result when both boxes are destroyed? For a deep-copy fix, name the two copy operations that must each create an independent owned integer. No implementation or output prediction is required.

```
class Box {
    int* p;
public:
    Box(int n) : p(new int(n)) {}
    ~Box() { delete p; }
};
{
    Box a(7);
    Box b = a;
}
```

6. (5 pts) In C++, give the output. If `delete p;` is removed, is object 3 destroyed when the inner block ends? In RAII (resource acquisition is initialization), which releases an owned resource: the owner's construction, the owner's destruction, or a raw pointer going out of scope? Assume allocation succeeds.

```
struct Mark {
    int id;
    Mark(int n) : id(n) { std::cout << "+" << id << " "; }
    ~Mark() { std::cout << "-" << id << " "; }
};

void run() {
    Mark a(1);
    {
        Mark b(2);
        Mark* p = new Mark(3);
        delete p;
    }
}

int main() { run(); }
```

7. (4 pts) In C++, identify the deletion bug and fix `Expr`'s destructor declaration. After the fix, give the destructor order. Do not predict the original undefined behavior.

```
struct Expr {
    virtual int eval() const = 0;
    ~Expr() {}
};

struct Number : public Expr {
    int eval() const { return 3; }
    ~Number() { std::cout << "Number "; }
};

Expr* p = new Number;
delete p;
```

8. (6 pts) Draw the AST for the following arithmetic expression, then evaluate it:

$$2 * (7 - 3) + 6/2$$

9. (6 pts) In C++, fill both declarations for a templated function that accepts `std::vector<int>` or `std::vector<double>` without copying or modifying the vector. Assume small vectors and that `T()` is zero.

```
/* template declaration */
T sum(/* parameter named xs */) {
    T total = T();
    for (unsigned int i = 0; i < xs.size(); ++i)
        total = total + xs[i];
    return total;
}
```

Explain the purpose of `const` and `&` in your parameter. Is selecting `sum<int>` versus `sum<double>` an example of run-time virtual-table dispatch or compile-time template instantiation?

10. (6 pts) In Python, what does this program print, and why?

```
x = [1, 2]
y = x
x.append(3)
x = [9]
print(y)
print(x)
```

11. (7 pts) Let $e = \mathbf{a|b}$. Compute and simplify $D_{\mathbf{a}}(e)$, $D_{\mathbf{b}}(e)$, and $D_c(e)$. Then use nullability of the three results to say which one-character strings the expression accepts.

12. (5 pts) For this grammar, a, b, c are terminal tokens, S, A, B are nonterminals, and ϵ denotes the empty string:

$$S \rightarrow AB \quad A \rightarrow aA \mid \epsilon \quad B \rightarrow bB \mid c$$

$\text{First}(s)$ contains possible first tokens and includes ϵ only when s can derive the empty string. Compute $\text{First}(A)$, $\text{First}(B)$, $\text{First}(S)$, and $\text{First}(Ab)$. Briefly justify $\text{First}(S)$. Here Ab means the nonterminal A followed by the single terminal b .

13. (3 pts) Assume safe Rust code and sound libraries. Which statement best describes Rust's memory-safety guarantee? Choose one. Which additional error does that guarantee prevent: use-after-free, an incorrect arithmetic result, or deadlock?

- a. Every program finishes normally without panicking.
- b. The program cannot access freed storage or free the same allocation twice through safe operations, but it can still panic.
- c. The compiler proves that the algorithm gives the intended answer.
- d. Rust requires a tracing garbage collector to free every object.

14. (3 pts) True / False: If a formal language is finite, then it is also regular.

15. (3 pts) True or False: In C++, dynamic dispatch applies to all method calls.

16. (3 pts) True or False: A valid scalar `delete` of a non-null pointer to a live C++ object runs its destructor before releasing its storage.

17. (5 pts) In C++, give the output. In one sentence, explain whether `r = y` changes `x` or makes `r` refer to `y`.

```
int x = 4, y = 9;
int* p = &x;
int& r = x;
*p = 6;
r = y;
p = &y;
*p = 12;
std::cout << x << " " << y << " " << r << "\n";
```

18. (4 pts) In Rust, give the output and explain what `_` means inside a tuple pattern. If the initializer were changed to `(0, 0)`, what would print, and which arm would be used?

```
let pair = (0, 5);
let label = match pair {
    (0, _) => "vertical",
    (_, 0) => "horizontal",
    _ => "neither",
};
println!("{}", label);
```

19. (6 pts) Let $A = \{a, b, c\}$. Enumerate $\mathcal{P}(A)$. What is the general formula for $|\mathcal{P}(X)|$, based on $|X|$?

20. (3 pts) Give the output of this Rust code. Why does the declaration of `x` need `mut`? What would happen at `x += 3` if `mut` were removed?

```
let mut x: i32 = 2;
x += 3;
let y = if x > 4 { x * 2 } else { 0 };
println!("{}", x, y);
```

21. (4 pts) This Rust function is intended to return one more than its argument:

```
fn next(n: i32) -> i32 {
    n + 1;
}
```

Does it compile as written? Explain the problem, give a one-character repair, and give an alternative repair using `return`. What does `-> i32` specify?

22. (3 pts) True / False: Encapsulation groups related state and operations behind an interface so that other code need not depend on every implementation detail.

23. (3 pts) True / False: Two languages can use similar syntax for a construct while giving that construct different semantics.

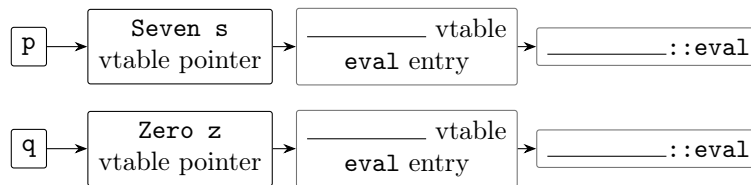
24. (3 pts) True / False: Python is dynamically typed: a name can be rebound to objects of different types, while each object still has a type.

25. (5 pts) In C++, consider these classes and calls:

```
struct Term {
    virtual int eval() const = 0;
    virtual ~Term() {}
};
struct Seven : public Term { int eval() const { return 7; } };
struct Zero : public Term { int eval() const { return 0; } };
Seven s; Zero z;
const Term* p = &s;
const Term* q = &z;
std::cout << p->eval() << " " << q->eval() << "\n";
```

Give the output. Next, suppose we add one new line of client code, not shown above, declaring `Term e;` — would that new line compile? Identify the declaration in the listing above that determines your answer.

Under the usual vtable implementation in C++, fill each blank below with **Seven** or **Zero**. Neither class declares a data member of its own, so the only data each object needs to store is its vtable pointer.



Name: _____

CptS 355 Exam 2 (Fall 2026)

Write your full name on every page. Keep explanations concise; partial credit is available. Assume required headers and enclosing functions are supplied for C++; Rust statement excerpts are inside `main`. Assume the usual vtable implementation in C++, and safe Rust with sound libraries.

1. (10 pts) In C++, write the recursive descent parsing function `int parseValue()` for this grammar:

$$V \rightarrow n \mid (V)$$

Here n represents a natural-number token. The function consumes one V and returns its numeric value, so parsing `((7))` returns 7. Use these supplied helpers:

```
std::string peek(); // next token's exact text; "" at end of input
bool isNat(const std::string& token); // true for one or more digits
int readNat(); // consume a natural-number token and return its value
void expect_token(const std::string& text); // consume text or report error
void syntaxError(); // report a syntax error; does not return
```

For example, `peek()` returns `"7"`, `"("`, or `)"`. Use `isNat` to recognize a number. Consume the matching parentheses around a recursively parsed V , and reject other starting tokens. All numbers fit in an `int`; the caller checks for leftover input.

Solution / grading guide. One suitable function is:

```
int parseValue() {
    if (isNat(peek())) return readNat();
    if (peek() == "(") {
        expect_token("(");
        int value = parseValue();
        expect_token(")");
        return value;
    }
    syntaxError();
    return 0; // unreachable; no value is returned on an error
}
```

Award 3 points for recognizing, consuming, and returning a number; 1 for recognizing and consuming `"("`; 2 for the recursive call; 2 for requiring and consuming `)"`; 1 for returning the inner value; and 1 for rejecting another starting token. Equivalent if/else code is valid. The final return is unreachable after `syntaxError()`.

2. (5 pts) Consider these Rust snippets independently. Which compiles, and which is rejected? State the borrowing rule involved. For the rejected snippet, describe how to perform the two increments one after the other without keeping both mutable borrows active together.

```
// A
let n = 1;
let a = &n;
let b = &n;
println!("{}", a, b);
```

```
// B
let mut n = 1;
let a = &mut n;
let b = &mut n;
*a += 1;
*b += 1;
println!("{}", n);
```

Solution / grading guide. A compiles: multiple shared borrows may coexist. B is rejected at the second mutable borrow, because `a` must still be used afterward. A mutable borrow requires exclusive access to the same value while it is active. One simple repair is:

```
{ let a = &mut n; *a += 1; }
{ let b = &mut n; *b += 1; }
println!("{}", n);
```

This repaired B prints 3. Moving the first increment before the second borrow also works when the first reference has no later uses.

Award 2 points for classification, 2 for the shared/exclusive rule, and 1 for a sequential-borrow repair. Do not demand lexical blocks as the only possible repair or assert that every borrow lasts to the end of its scope.

3. (4 pts) In C++, explain when `p` becomes dangling and why printing 7 is not guaranteed. Repair the function and caller by returning an integer by value, without heap allocation. Do not predict undefined behavior.

```
int* makeNumber() {
    int n = 7;
    return &n;
}
int* p = makeNumber();
std::cout << *p << "\n";
```

Solution / grading guide. `n` has automatic storage duration: its lifetime ends when `makeNumber` returns. The returned address is dangling, and accessing the expired integer through `*p` has undefined behavior. A safe repair is:

```
int makeNumber() { return 7; }
int n = makeNumber();
std::cout << n << "\n";
```

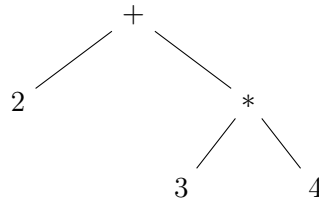
Award 2 points for the lifetime/dangling diagnosis and 2 for a correct return-by-value repair. Do not require a particular physical stack layout.

4. (7 pts) Prepare this grammar for recursive descent parsing; n is a natural-number token:

$$E \rightarrow E + T \mid E - T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow n \mid (E)$$

Identify both immediately left-recursive nonterminals. Suppose you wrote a recursive descent parsing function `parseE()` directly from the rule for E . Why would its first recursive call fail to make progress? Remove E 's immediate left recursion using one new nonterminal; do not rewrite T . Draw the AST for $2 + 3 * 4$ (no grammar-only nodes) and give its value.

Solution / grading guide. Both E and T are immediately left recursive. The initial `parseE()` call would call itself again without consuming a token, causing unbounded recursion. A suitable rewrite is $E \rightarrow TS$, $S \rightarrow +TS \mid -TS \mid \epsilon$. The AST is:



Its value is 14. Equivalent operator labels are valid.

Award 1 point for identifying both nonterminals, 1 for the no-progress diagnosis, 2 for the rewrite, 2 for the AST/precedence, and 1 for the value. A left-associative AST builder can fold the rewritten repetition into the accumulated left subtree.

5. (7 pts) In C++, each `Box` owns its allocated integer. After the default copy `Box b = a`, do the two pointers designate the same allocation or different allocations? What bug could result when both boxes are destroyed? For a deep-copy fix, name the two copy operations that must each create an independent owned integer. No implementation or output prediction is required.

```

class Box {
    int* p;
public:
    Box(int n) : p(new int(n)) {}
    ~Box() { delete p; }
};
{
    Box a(7);
    Box b = a;
}

```

Solution / grading guide. The generated copy constructor copies the pointer member, not the allocated integer: `a.p` and `b.p` designate the same allocation. `b` is destroyed first and frees it; `a`'s destructor then attempts to delete it again, causing undefined behavior.

A deep-copy policy gives each box its own integer. Both the copy constructor and copy-assignment operator must implement that policy; assignment also handles the old resource and self-assignment correctly.

Award 2 points for memberwise pointer copying, 1 for the shared target, 2 for double deletion, and 2 for naming both copy operations for deep copying.

6. (5 pts) In C++, give the output. If `delete p;` is removed, is object 3 destroyed when the inner block ends? In RAII (resource acquisition is initialization), which releases an owned resource: the owner's construction, the owner's destruction, or a raw pointer going out of scope? Assume allocation succeeds.

```
struct Mark {
    int id;
    Mark(int n) : id(n) { std::cout << "+" << id << " "; }
    ~Mark() { std::cout << "-" << id << " "; }
};

void run() {
    Mark a(1);
    {
        Mark b(2);
        Mark* p = new Mark(3);
        delete p;
    }
}

int main() { run(); }
```

Solution / grading guide. Ignoring the final space, the output is +1 +2 +3 -3 -2 -1. `delete` destroys the heap object immediately; the inner block then destroys `b`, and function exit destroys `a`. Without `delete`, object 3 leaks: the raw pointer's scope exit does not run its pointee's destructor. In RAII (resource acquisition is initialization), a constructor acquires a resource and the owning object's destructor releases it; an automatic owner's scope exit therefore performs cleanup.

Award 3 points for the order, 1 for the leak explanation, and 1 for selecting the owning object's destruction as the RAII cleanup event.

7. (4 pts) In C++, identify the deletion bug and fix `Expr`'s destructor declaration. After the fix, give the destructor order. Do not predict the original undefined behavior.

```
struct Expr {
    virtual int eval() const = 0;
    ~Expr() {}
};

struct Number : public Expr {
    int eval() const { return 3; }
    ~Number() { std::cout << "Number "; }
};

Expr* p = new Number;
delete p;
```

Solution / grading guide. The base destructor is nonvirtual. Deleting this `Number` through `Expr*` therefore has undefined behavior, even though `eval` is virtual. Change the declaration to:

```
virtual ~Expr() {}
```

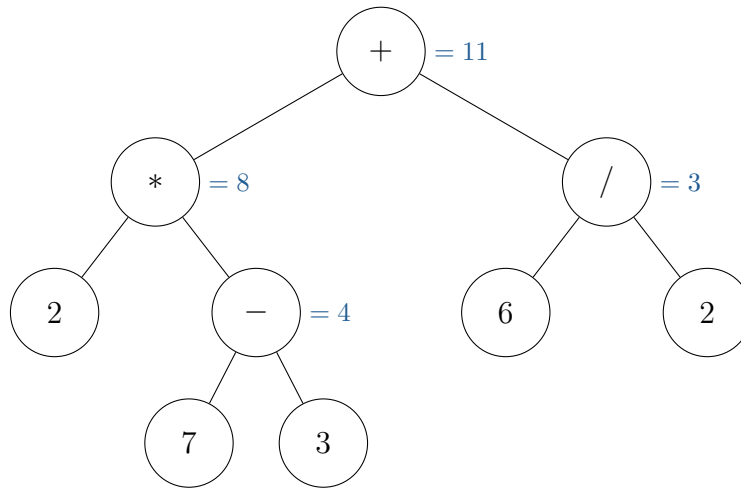
Then `Number`'s destructor runs first (printing `Number`), followed by `Expr`'s destructor; the allocation is released afterward.

Award 1 point for the diagnosis, 2 for the virtual base destructor, and 1 for derived-before-base destruction. Do not accept "it is guaranteed to run only the base destructor" as a description of the original code.

8. (6 pts) Draw the AST for the following arithmetic expression, then evaluate it:

$$2 * (7 - 3) + 6 / 2$$

Solution.



9. (6 pts) In C++, fill both declarations for a templated function that accepts `std::vector<int>` or `std::vector<double>` without copying or modifying the vector. Assume small vectors and that `T()` is zero.

```
/* template declaration */
T sum(/* parameter named xs */) {
    T total = T();
    for (unsigned int i = 0; i < xs.size(); ++i)
        total = total + xs[i];
    return total;
}
```

Explain the purpose of `const` and `&` in your parameter. Is selecting `sum<int>` versus `sum<double>` an example of run-time virtual-table dispatch or compile-time template instantiation?

Solution / grading guide. Use `template<class T>` (or `template<typename T>`) and `const std::vector<T>& xs`. The reference avoids copying the vector; `const` prevents modifying it through this parameter and also permits a const vector argument. The compiler instantiates the template for the chosen element type. This is static generic polymorphism; it needs no virtual table to select the `int` versus `double` version.

Award 2 points for the template declaration, 2 for the parameter, 1 for its `const`/reference meaning, and 1 for compile-time instantiation.

10. (6 pts) In Python, what does this program print, and why?

```
x = [1, 2]
y = x
x.append(3)
x = [9]
print(y)
print(x)
```

Solution / grading guide.

```
[1, 2, 3]
[9]
```

Initially `x` and `y` point to the same mutable list, so `append` is visible through both names. The last assignment rebinds only `x` to a new list; it does not change the object still referenced by `y`.

11. (7 pts) Let $e = a|b$. Compute and simplify $D_a(e)$, $D_b(e)$, and $D_c(e)$. Then use nullability of the three results to say which one-character strings the expression accepts.

Solution / grading guide. Derivatives distribute over disjunction:

$$\begin{aligned}D_a(a|b) &= \epsilon|\emptyset = \epsilon, \\D_b(a|b) &= \emptyset|\epsilon = \epsilon, \\D_c(a|b) &= \emptyset|\emptyset = \emptyset.\end{aligned}$$

The first two residuals are nullable and the last is not, so the accepted one-character strings are exactly `a` and `b`.

12. (5 pts) For this grammar, a, b, c are terminal tokens, S, A, B are nonterminals, and ϵ denotes the empty string:

$$S \rightarrow AB \quad A \rightarrow aA \mid \epsilon \quad B \rightarrow bB \mid c$$

$\text{First}(s)$ contains possible first tokens and includes ϵ only when s can derive the empty string. Compute $\text{First}(A)$, $\text{First}(B)$, $\text{First}(S)$, and $\text{First}(Ab)$. Briefly justify $\text{First}(S)$. Here Ab means the nonterminal A followed by the single terminal b .

Solution / grading guide. The sets are $\text{First}(A) = \{a, \epsilon\}$, $\text{First}(B) = \{b, c\}$, $\text{First}(S) = \{a, b, c\}$, and $\text{First}(Ab) = \{a, b\}$. Because A can disappear, a string generated by AB can begin with a token from B . But B cannot disappear, so AB is not nullable. Likewise the terminal b in Ab cannot disappear.

Award 1 point per correct set and 1 for explaining both the nullable prefix and why the whole string cannot be empty. Do not treat ϵ as an input token to consume.

13. (3 pts) Assume safe Rust code and sound libraries. Which statement best describes Rust's memory-safety guarantee? Choose one. Which additional error does that guarantee prevent: use-after-free, an incorrect arithmetic result, or deadlock?

- a. Every program finishes normally without panicking.
- b. The program cannot access freed storage or free the same allocation twice through safe operations, but it can still panic.
- c. The compiler proves that the algorithm gives the intended answer.
- d. Rust requires a tracing garbage collector to free every object.

Solution / grading guide. Choice (b). Ownership and borrowing prevent accessing freed storage or freeing the same allocation twice through safe operations. The additional error is use-after-free. A panic, incorrect arithmetic result, or deadlock does not itself violate memory safety; the guarantee is not a proof of algorithmic correctness or termination.

Award 2 points for (b) and 1 for use-after-free.

14. (3 pts) True / False: If a formal language is finite, then it is also regular.

True

15. (3 pts) True or False: In C++, dynamic dispatch applies to all method calls.

False

16. (3 pts) True or False: A valid scalar `delete` of a non-null pointer to a live C++ object runs its destructor before releasing its storage.

True

17. (5 pts) In C++, give the output. In one sentence, explain whether `r = y` changes `x` or makes `r` refer to `y`.

```
int x = 4, y = 9;
int* p = &x;
int& r = x;
*p = 6;
r = y;
p = &y;
*p = 12;
std::cout << x << " " << y << " " << r << "\n";
```

Solution / grading guide. The output is 9 12 9. The reference `r` remains an alias for `x`. Assigning `r = y` copies the value 9 into `x`; it does not rebind the reference. Reassigning `p` changes which object the pointer designates, so the final dereference modifies `y`.

Award 3 points for the final values and 2 for explaining assignment through a reference versus pointer reassignment.

18. (4 pts) In Rust, give the output and explain what `_` means inside a tuple pattern. If the initializer were changed to `(0, 0)`, what would print, and which arm would be used?

```
let pair = (0, 5);
let label = match pair {
    (0, _) => "vertical",
    (_, 0) => "horizontal",
    _ => "neither",
};
println!("{}", label);
```

Solution / grading guide. The output is `vertical`. In `(0, _)`, the first component must be zero and the second may be any value. For `(0, 0)`, the output is still `vertical`: Rust chooses the first matching arm, even though the second arm would also match. The final `_` accepts any remaining pair.

Award 1 point for the original output, 1 for the tuple wildcard, 1 for the changed output, and 1 for first-matching-arm selection.

19. (6 pts) Let $A = \{a, b, c\}$. Enumerate $\mathcal{P}(A)$. What is the general formula for $|\mathcal{P}(X)|$, based on $|X|$?

Solution / grading guide.

$$\mathcal{P}(A) = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, A\}.$$

There are eight subsets, so $|\mathcal{P}(A)| = 8 = 2^3$. In general, $|\mathcal{P}(A)| = 2^n$ when $|A| = n$: each element independently has two choices, present or absent.

20. (3 pts) Give the output of this Rust code. Why does the declaration of `x` need `mut`? What would happen at `x += 3` if `mut` were removed?

```
let mut x: i32 = 2;
x += 3;
let y = if x > 4 { x * 2 } else { 0 };
println!("{}", x, y);
```

Solution / grading guide. The output is 5 10. An `if` is an expression here: the selected branch supplies `y`'s value. Rust bindings are immutable by default; `mut` allows assigning a new value to `x`. Without it, the assignment `x += 3` is a compile-time error.

Award 2 points for the values and 1 for the mutability/compile-error explanation. No ownership or lifetime reasoning is needed.

21. (4 pts) This Rust function is intended to return one more than its argument:

```
fn next(n: i32) -> i32 {
    n + 1;
}
```

Does it compile as written? Explain the problem, give a one-character repair, and give an alternative repair using `return`. What does `-> i32` specify?

Solution / grading guide. It does not compile. The semicolon discards the expression's value, leaving the body with unit type `()`, which conflicts with the declared `i32` return type. Remove the semicolon to use the final expression `n + 1`, or write `return n + 1;`. The arrow specifies the function's return type.

Award 1 point for the type mismatch, 1 for removing the semicolon, 1 for an explicit return, and 1 for the arrow's meaning. Explaining that the body no longer returns the required integer is enough without naming unit.

22. (3 pts) True / False: Encapsulation groups related state and operations behind an interface so that other code need not depend on every implementation detail.

True

23. (3 pts) True / False: Two languages can use similar syntax for a construct while giving that construct different semantics.

True

24. (3 pts) True / False: Python is dynamically typed: a name can be rebound to objects of different types, while each object still has a type.

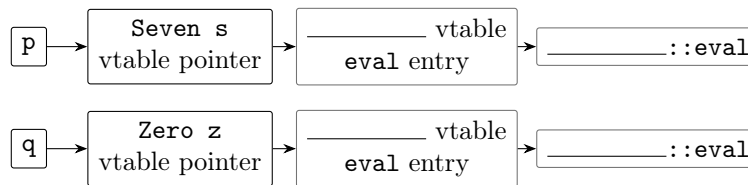
True

25. (5 pts) In C++, consider these classes and calls:

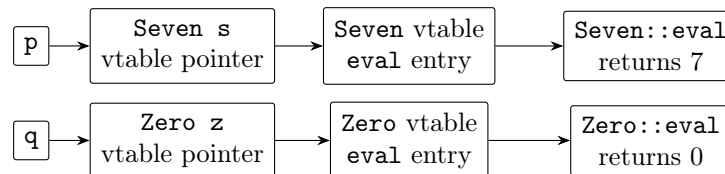
```
struct Term {
    virtual int eval() const = 0;
    virtual ~Term() {}
};
struct Seven : public Term { int eval() const { return 7; } };
struct Zero : public Term { int eval() const { return 0; } };
Seven s; Zero z;
const Term* p = &s;
const Term* q = &z;
std::cout << p->eval() << " " << q->eval() << "\n";
```

Give the output. Next, suppose we add one new line of client code, not shown above, declaring `Term e;` — would that new line compile? Identify the declaration in the listing above that determines your answer.

Under the usual vtable implementation in C++, fill each blank below with `Seven` or `Zero`. Neither class declares a data member of its own, so the only data each object needs to store is its vtable pointer.



Solution / grading guide. The output is 7 0. The new line `Term e;` is rejected because `virtual int eval() const = 0;` is pure virtual, so `Term` is abstract: no `Term` object can be created. The pointers `p` and `q` are still fine, since a pointer to an abstract class need only point at a concrete derived object. The four blanks are `Seven`, `Seven`, `Zero`, and `Zero`, respectively:



Only the `eval` entries are shown.

Award 1 for the output, 1 for rejecting `Term e;`, 1 for naming the pure virtual method, and $\frac{1}{2}$ per blank.